

FlakeHunter

An AI agent that finds flaky CI jobs, proves why, and opens a pull request to quarantine them.

EXASOL AI + DATA CHALLENGE 2026

Find

Identify flaky CI jobs using statistical patterns across real run data

Prove

Generate and test hypotheses with SQL queries on Exasol — no invented evidence

Act

Open a pull request to quarantine the flaky job with database-backed evidence

The Problem: Flaky Tests Waste Hours

What is a flaky test?

A flaky test fails on **correct code** — not because of a real bug, but due to timing, infrastructure, or non-determinism. The code hasn't changed. The test result has.

The consequences compound

Wasted investigation time

Engineers spend ~1 hour chasing bugs that don't exist — re-running pipelines, bisecting commits, reading logs for a failure that was never real.

Alert fatigue sets in

Teams start ignoring red CI. Once red becomes the default, the signal is gone.

Real failures slip through

When nobody trusts the CI, genuine regressions go undetected. The cost of a flaky test isn't just the wasted hour — it's the real bug that hides behind it.

The Signal: Same Commit, Different Results

The core insight behind FlakeHunter is simple: **if the same commit both passes and fails, the code isn't the cause**. Flakiness is a property of the job, not the change.

Real example — pytorch/pytorch

One commit ran **84 times** across CI infrastructure.

82

Passed

2

Failed

84

Total runs

Why this pattern matters

A job that fails 2 out of 84 times on the same commit is not broken — it is **flaky**. The distinction is critical:

Flaky pattern

pass → fail → pass — the same commit produces inconsistent results. The code is not the variable.

Broken pattern

pass → fail → fail — results are consistent after the change. A real regression has occurred.

Window functions in Exasol allow FlakeHunter to distinguish these two patterns precisely across the full dataset.

How the Agent Works

FlakeHunter runs a structured investigative loop — generating hypotheses, writing SQL to test them, and refusing to state any number that isn't in a query result.



Triage

Identify candidate flaky jobs from the dataset of **21,654 real CI jobs** using statistical patterns.



Probe

Write SQL for each hypothesis and execute on Exasol via **24 MCP tools** for SQL execution and schema introspection.



Loop

Refine the hypothesis and re-probe until the agent is confident or has exhausted the available evidence.



Hypothesize

Generate theories for the failure mode: timeout, race condition, infrastructure variance, or real regression.



Assess

Attempt to *disprove* the leading hypothesis — not confirm it. Failed queries can never count as evidence.



Conclude

Deliver a verdict with a **confidence score** grounded entirely in query results from the database.

A Real Investigation: `get_workflow_conclusion`

FlakeHunter investigated the `get_workflow_conclusion` job from the pytorch/pytorch dataset. The agent generated three hypotheses and systematically ruled out two of them.

Runtime evidence

4.01s

Passing avg

Average duration of successful runs

34.5s

Failing avg

Average duration of failed runs

8.6×

Slower on fail

Failing runs take 8.6× longer than passing runs

An 8.6× runtime gap between passing and failing runs is a strong signal of a **timeout** — the job runs until it hits a wall, not until it completes.

Hypothesis scorecard

✓ Timeout

Supported. 8.6× runtime gap on failing runs. Leading hypothesis confirmed.

❑ Runner infrastructure

Ruled out. No correlation between runner identity and failure rate.

❑ Real regression

Ruled out. Same commit produces both pass and fail — code is not the variable.

Verdict

Flaky — confidence 0.85. Recommend quarantine via `continue-on-error: true`.

Why Exasol: Speed at Scale

FlakeHunter's investigative loop requires many SQL queries in rapid succession. Exasol's columnar engine makes this tractable — even for non-equi self-joins that generate hundreds of millions of comparisons.

0.07s

Flake triage

Full triage across all jobs in the dataset

0.34s

Self-join

Non-equi self-join over 13,331 jobs (~178M comparisons)

21,654

CI jobs

Real CI jobs analysed from pytorch/pytorch

24

MCP tools

Tools for SQL execution and schema introspection

The self-join problem

Detecting flakiness requires comparing every run of a job against every other run of the same job on the same commit. For 13,331 jobs, this produces approximately **178 million comparisons**. Exasol completes this in **0.34 seconds**.

Window functions for pattern detection

Exasol window functions allow FlakeHunter to distinguish the **pass→fail→pass** (flaky) pattern from the **pass→fail→fail** (broken) pattern across the full job history — without loading data into Python or post-processing results.

What We Got Wrong (and Fixed)

The first version of the agent was confidently wrong. It invented evidence, fabricated runner names, and reported failure rates from queries that had errored out. Here is what happened and how it was fixed.

The failure mode

Invented evidence

The agent fabricated runner names and failure rates that did not exist in the database.

Errored queries counted

Queries that returned errors were treated as supporting evidence. Confidence reached 0.95 from data that was never retrieved.

No schema grounding

The agent guessed column names and table structures, producing plausible-sounding but invalid SQL.

The three fixes

1 Failed queries are not evidence

A query that errors, times out, or returns zero rows cannot be cited in the verdict. The agent must re-probe or acknowledge the gap.

2 Schema delivered upfront

The agent receives the real schema before it writes a single query. No guessing column names or table structures.

3 Numbers must come from results

The agent may not state any number that isn't present in a query result. Every figure in the verdict is traceable to a specific SQL output.

The Action: Quarantine via Pull Request

flakehunter act translates the agent's verdict into a concrete code change — a pull request on the user's own fork, never upstream.

What the PR does

Adds `continue-on-error: true`

The flaky job is quarantined: it still runs, but a failure no longer blocks the pipeline.

Evidence from the database

The PR description is populated with query results — not with AI-generated text. Every number cited is traceable to a SQL output.

Honest about scope

The PR clearly states: quarantine does not fix the root cause. It masks the symptom so the team can ship while investigating properly.

Safety constraints

Fork only

The agent opens PRs on the user's own fork. It never touches the upstream repository.

Minimal change

1 file changed, +3 lines. The quarantine is surgical — no refactoring, no collateral edits.

Human in the loop

The PR is a proposal. A human reviews and merges. FlakeHunter does not auto-merge.

Proof: Live Pull Request

FlakeHunter opened a real pull request against a fork of pytorch/pytorch. The change is minimal, the evidence is database-backed, and the PR is publicly visible.

PR details

Location

`github.com/mdfaheem23/pytorch/pull/1`

Change size

1 file changed, +3 lines —
adds `continue-on-error: true` to the `get_workflow_conclusion` job definition.

Evidence sourcing

All numbers in the PR description (4.01s, 34.5s, 8.6×, confidence 0.85) are pulled directly from Exasol query results — not generated by the AI.

Scope statement

PR explicitly notes that quarantine masks the symptom; root cause investigation is still required.

Screenshot: PR diff and evidence

 Screenshots of the live PR diff and evidence block will be inserted here by the author. See `github.com/mdfaheem23/pytorch/pull/1` for the live version.

The PR demonstrates the full FlakeHunter pipeline end-to-end: from raw CI data in Exasol, through the agent's investigative loop, to a concrete, reviewable code change on GitHub.

Lessons from Exasol Personal

Five concrete integration issues encountered during development — each resolved, each worth documenting for anyone building on the Exasol MCP stack.

1

SQL execution off by default

The MCP server ships with SQL execution disabled. It must be explicitly enabled in configuration before the agent can run any queries.

2

Self-signed certificate

The self-signed certificate breaks the MCP connection until certificate validation is disabled. Not obvious from the default error message.

3

CSV HTTP import

CSV import via HTTP cannot reach the host from inside the VM. A different import path is required — this blocked initial data loading.

4

pyexasol placeholder syntax

pyexasol uses `{name}` placeholders, not the `?` syntax common in other Python database drivers. Mixing them produces silent failures.

5

CREATE OR REPLACE TABLE wipes data

`CREATE OR REPLACE TABLE` in a re-applied schema drops and recreates the table, deleting all existing data. Use `CREATE TABLE IF NOT EXISTS` for idempotent schema application.

Team Members

Mohammed Faheem K

Rakesh N

Sanjeev Kumar KS

Sanjay Ramaswamy

Gowrish

Vellore Institute of Technology